



INSA ROUEN NORMANDIE

Département Informatique

---

# PROJET INTÉGRATIF SMART ROBOT

Robot Autonome

---

## Équipe projet :

Delacroix-Henrion Mathieu

Diallo Alioune

Malherbe Ylann

Mathieu Ilan

Olivier Neil

Schetrit Jahnis

ITI3 — Année 2025-2026

**Décembre 2025**

# Table des matières

|          |                                                            |          |
|----------|------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                        | <b>2</b> |
| 1.1      | Contexte . . . . .                                         | 2        |
| 1.2      | Problématique . . . . .                                    | 2        |
| <b>2</b> | <b>Partie électronique</b>                                 | <b>3</b> |
| 2.1      | Conception du Robot . . . . .                              | 3        |
| 2.1.1    | Schéma électrique . . . . .                                | 3        |
| 2.1.2    | Choix techniques . . . . .                                 | 3        |
| 2.1.3    | Estimation du robot . . . . .                              | 3        |
| 2.2      | Choix des composants . . . . .                             | 3        |
| 2.2.1    | Capteur suiveur de ligne . . . . .                         | 4        |
| 2.2.2    | Capteur RGB . . . . .                                      | 4        |
| 2.2.3    | Moteur Shield . . . . .                                    | 4        |
| 2.2.4    | Ecran LCD . . . . .                                        | 5        |
| 2.2.5    | Buzzer . . . . .                                           | 5        |
| 2.3      | Cahier des charges . . . . .                               | 5        |
| 2.4      | Difficultés rencontrées et pistes d'amélioration . . . . . | 5        |
| <b>3</b> | <b>Partie informatique</b>                                 | <b>5</b> |
| 3.1      | Cycle en V . . . . .                                       | 5        |
| 3.1.1    | Analyse . . . . .                                          | 5        |
| 3.1.2    | Conception préliminaire . . . . .                          | 6        |
| 3.1.3    | Conception détaillée . . . . .                             | 8        |
| 3.1.4    | Code C . . . . .                                           | 8        |
| 3.2      | Conclusion . . . . .                                       | 8        |
| 3.2.1    | Répartition du travail . . . . .                           | 8        |
| 3.2.2    | Conclusion du projet . . . . .                             | 8        |

# 1 Introduction

## 1.1 Contexte

Dans le cadre de notre premier semestre dans le département ISIA, nous avons pour but de réaliser un Smart Robot capable de se déplacer dans une ville modélisée sous forme de cases. Equipés de capteurs, moteurs et d'une raspberry Pi 4, notre objectif est de construire et coder un robot capable de parcourir un itinéraire imposés tout en déterminant le plus court chemin permettant de passer par un certain nombre de points obligatoires. C'est en combinant électronique, algorithmique et développement en C, ainsi que l'utilisation de GitLab que nous allons répondre aux attentes de ce projet. Ce rapport est structuré de la manière suivante : Dans un premier temps, nous présenterons la partie électronique contenant la description de la conception du robot avec le schéma électrique, les choix techniques que nous avons effectués, ainsi que les branchements et problèmes que nous avons pu rencontrés dans cette partie.

Dans un deuxième temps, nous présenterons la partie informatique, depuis l'analyse descendante du problème, jusqu'à l'implémentation des fonctions et procédures en C. Enfin, nous concluerons sur ce projet avec la répartition des tâches et ce que l'on a pu tirer de ce projet.

## 1.2 Problematique

La problématique principale de ce projet est de faire en sorte que le robot parcoure la ville de manière autonome et optimale en interprétant correctement les descriptions de la ville. En effet, le robot doit lire et traduire ces informations en une suite d'ordre (AV, TG, TD) lui permettant de se déplacer dans la ville. Celui-ci doit non seulement suivre les lignes au sol en faisant attention à corriger d'éventuelles déviations, mais aussi détecter et signaler les pastilles colorées signalant les points de passages obligatoires. En définitive, notre robot doit être capable de passer par les trois points obligatoires repérables par des pastilles, afficher sur l'écran la couleur de celle-ci tout en buzzant, afficher chaque intersection qu'il croise, ainsi que chaque action qu'il réalise tout le long du parcours (Avancer, Tourner Gauche, Tourner Droite).

## 2 Partie électronique

### 2.1 Conception du Robot

#### 2.1.1 Schéma électrique

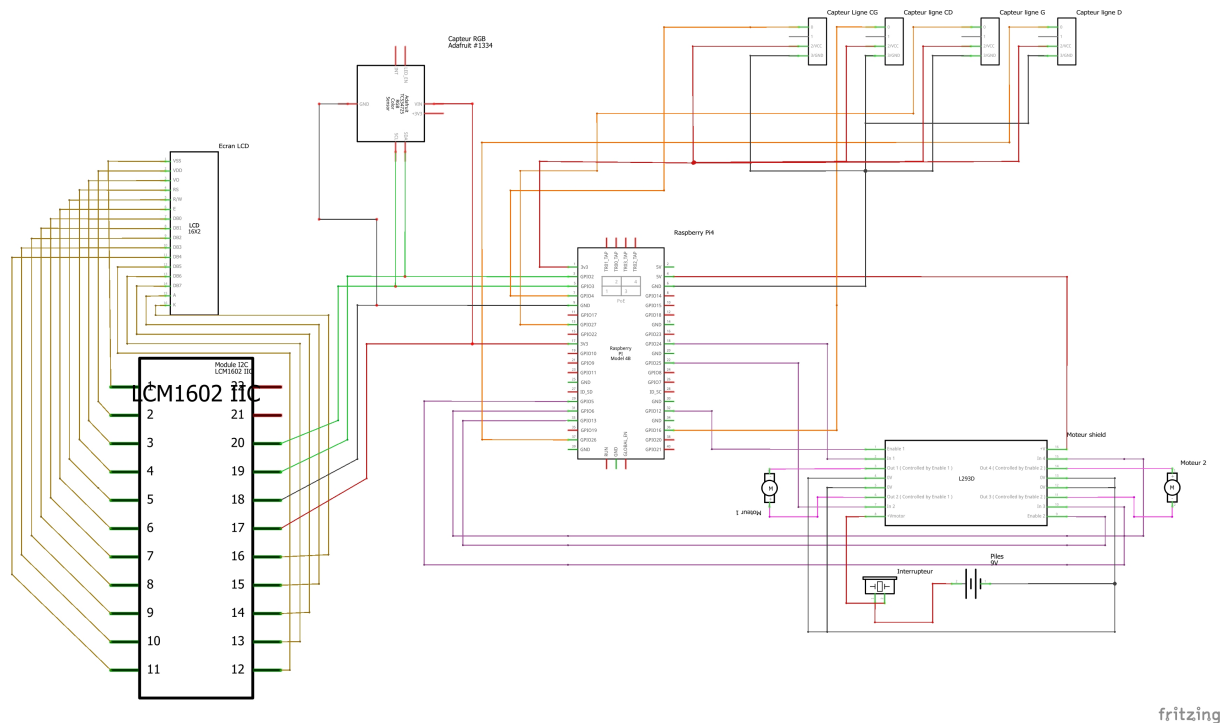
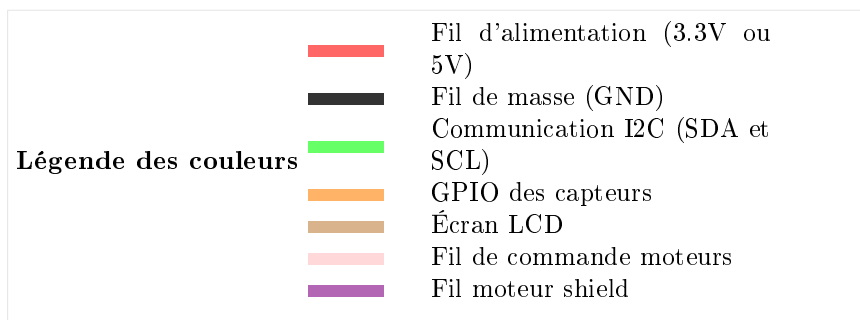


FIGURE 1 – Schéma électrique du robot autonome



#### 2.1.2 Choix techniques

#### 2.1.3 Estimation du robot

### 2.2 Choix des composants

Pour le montage du robot, nous avons commencé par nous rassembler et discuter des différents composants que nous pouvions inclure au robot. Tout d'abord, nous avons discuté du meilleur emplacement pour les capteurs, essentiels au projet pour détecter les lignes noires du parcours. Après plusieurs propositions, nous avons finalement décidé de positionner deux capteurs proches l'un de l'autre en avant pour pouvoir suivre correctement la ligne noire et alors détecter par la même occasion lorsque le robot dévie. De plus, nous avons choisi de placer deux capteurs sur les cotés, un capteur sur le côté droit, ainsi qu'un capteur sur le côté gauche afin de détecter les intersections et virages. En ce qui concerne les moteurs, ceux-ci sont alimentés à l'aide de piles connectées en parallèles et sont reliés à un interrupteur. La raspberry Pi est alimentée par

une batterie externe afin de garantir une autonomie et une alimentation stable. Le buzzer et les capteurs (suiveurs de ligne et RGB) sont connectés aux GPIO de la RaspberryPi et alimentés avec 3,3V. Concernant l'écran LCD, celui-ci nécessite une alimentation de 5V comme précisé dans sa documentation technique. Avant de monter intégralement les composants sur le robot, nous avons testés chacun des composants séparément : Une unité de test pour les moteurs (Avancer, reculer, s'arrêter...), une unité de test pour les capteurs (détecter une surface sombre, détecter une surface claire, détecter une pastille de couleur...), ainsi qu'une unité de test pour l'affichage sur l'écran LCD (afficher 'Bonjour', effacer l'écran, afficher la couleur détectée par le capteur RGB...). Ces test nous ont permis de d'ajuster l'emplacement des composants, la calibration des capteurs, de la même manière que trouver plus facilement ce qui pouvait ne pas fonctionner.

### 2.2.1 Capteur suiveur de ligne

#### Rôle

Les capteurs suiveurs de ligne permettent au robot de détecter et donc de suivre une ligne noire tracée sur une surface plus claire. L'objectif est de guider les mouvements du robot, de telle sorte que celui-ci corrige sa direction s'il n'est pas bien aligné sur la trajectoire de la ligne noire. En l'incluant dans notre projet, ce type de capteur nous permet de garder le robot sur la ligne noir, de détecter lorsque celui-ci dévie de la trajectoire, mais cela permet aussi de repérer les intersections et virages lui permettant de s'orienter.

#### Fonctionnement

Ces types de capteurs fonctionnent de la manière suivante :  
Le noir absorbe la lumière tandis que le blanc ou les couleurs plus claires la réfléchisse. Les capteurs de ligne sont alors composés d'une LED infrarouge qui éclaire le sol en continu et d'une photodiode mesurant la lumière renvoyée. Concrètement, si un capteur suiveur de ligne se trouve sur une surface claire, alors la lumière infrarouge est fortement réfléchi. Le capteur reçoit donc beaucoup de lumière et envoie le signal indiquant qu'il s'agit d'une surface claire. Au contraire, sur une surface noire, la lumière est absorbée donc le capteur ne reçoit que très peu de lumière. Celui-ci envoie alors le signal indiquant qu'il s'agit d'une surface noire.

### 2.2.2 Capteur RGB

#### Rôle

Le capteur de couleur TCS 34725 permet au robot de détecter la couleur des pastilles présentes sur le parcours comme la pastille verte indiquant le point de départ du robot. Ce capteur est très important car il permet d'identifier correctement les points de passage, mais aussi d'afficher la couleur que ce capteur détecte sur l'écran LCD. Il est essentiel d'inclure ce capteur, sans quoi, notre robot ne serait pas capable de repérer les points de passages obligatoires du parcours.

#### Fonctionnement

Le capteur de RGB, comme son nom l'indique, permet de repérer trois couleurs : rouge, vert et bleu. Le TCS 34725 possède une petite LED blanche pour éclairer la surface et alors mesurer l'intensité lumineuse pour chacune des composantes (composantes rouge, ...). Ensuite, les valeurs sont converties en valeurs numériques via le moyen de communication I2C qui sont ensuite lus par la carte embarquée de la Raspberry Pi 4. Les trois intensités sont alors combinées pour déterminer de quelle couleur il s'agit.

### 2.2.3 Moteur Shield

#### Rôle

Le moteur driver est un intermédiaire entre la Raspberry Pi 4 et les moteurs du robot car il permet de fournir assez de courant aux moteurs, régler la vitesse, contrôler le sens de rotation des roues mais aussi de protéger la raspberry car celle ci ne peut pas alimenter directement les moteurs. En effet, la Raspberry ne fournirait que 3.3V ou au maximum 5V pour les moteurs

tandis que ceux-ci peuvent être alimentés jusqu'à 12V. Il est donc indispensable d'utiliser un moteur shield.

### **Fonctionnement**

#### **2.2.4 Ecran LCD**

##### **Rôle**

Un écran LCD I2C permet d'afficher les informations provenant du robot comme les indications concernant son trajet sur le parcours (Tourner gauche, intersection. . .) Mais aussi d'afficher la couleur des pastilles situées sur les points de passage obligatoires. Concrètement, il s'agit de l'interface entre le robot et nous, utilisateur.

##### **Fonctionnement**

Un écran LCD utilise beaucoup de fils, pour gérer cela, nous utilisons un LCD avec l'I2C pour réduire à seulement 2 fils de communication. Le câblage est alors plus simple et cette configuration est idéale pour notre robot surtout quand on utilise déjà beaucoup de GPIO avec les capteurs et moteurs. Seulement 4 lignes sont utilisées :

- SDA : données
- SCL : horloge
- VCC : alimentation
- GND : masse

Il est intéressant de noter que tous les périphériques I2C sont connectés en parallèle sur SDA/SCL. C'est exactement ce que l'on a pu faire avec le capteur RGB. L'écran LCD fonctionne de la manière suivante : Le micro contrôleur envoie les commandes comme le texte à afficher ou encore effacer l'écran. Le module I2C lui, reçoit les données et les convertit en signaux compréhensibles par le LCD.

#### **2.2.5 Buzzer**

##### **Rôle Fonctionnement**

### **2.3 Cahier des charges**

### **2.4 Difficultés rencontrées et pistes d'amélioration**

Nous avons rencontré de nombreuses difficultés, aussi bien sur le plan électronique que sur le plan organisationnel. Tout d'abord,

## **3 Partie informatique**

### **3.1 Cycle en V**

#### **3.1.1 Analyse**

L'analyse a un rôle essentiel dans ce projet afin d'identifier les principaux problèmes que cause ce projet, et alors de traiter ces problèmes en sous problèmes plus simples à résoudre. Les principaux problèmes à résoudre sont :

- étudier le fichier **.txt** afin de récupérer les informations essentielles liées au robot et à la construction de la ville
- créer un graphe modélisant les liaisons entre les différents points de la ville
- calculer le chemin le plus court
- générer un fichier **.txt** contenant les ordres que le robot doit effectuer

## 1. Types

**Type Direction** Ce type permet d'associer une direction(Sud, Nord, Ouest, Est) au robot et permet donc de faciliter l'interdiction des demi-tours quant à la direction actuelle du robot.

**Type Ordre** Ce type est une énumération qui permet de donner au robot les actions qu'il peut effectuer tout au long du parcours (Avancer, Tourner Droite, Tourner Gauche, Ne Rien Faire).

**Type Robot** Ce type sera défini sous forme d'une structure permettant de contenir la case actuelle ou se trouve le robot, ainsi que sa direction.

**Type Chemin** Ce type permettra de contenir l'ensemble des points par lequel le robot doit passer pour réaliser le chemin le plus court.

**Type EtatDuDijkstra** Ce type sera représenté par une structure permettant de renseigner le point actuel ou l'on se trouve, le point depuis lequel on arrive (point précédent) ainsi que le chemin pour parvenir jusqu'au point actuel.

**Type Solution** Ce type sera utilisé lors de la résolution du chemin le plus court, regroupant alors l'ordre dans lequel nous allons passer par les points obligatoires ainsi que le chemin complet des points par lesquels nous allons passer.

**Type File** Le dernier type que nous avons décidé d'implémenter est une structure File qui stockera les différents états du Dijkstra que l'on pourra obtenir grâce aux indices de début et fin de la file.

Pour ce qui est du Graphe, nous utilisons les TAD déjà implémentés de Monsieur Delestre utilisant de la même manière les TAD ListeChaineListe ainsi que tableHachage.

## 3. Analyse descendante

Une fois avoir pré-défini les types et structure que nous souhaitons mettre en place, nous avons étudié les algorithmes de parcours afin de choisir le plus adapté. Souhaitant représenter la ville sous forme de graphe, nous avons le choix entre deux algorithmes bien précis :

- **Dijkstra** : un algorithme efficace pour trouver le plus court chemin dans un graphe pondéré.
- **A\*** : une version améliorée de Dijkstra, utilisant une heuristique pour optimiser la recherche.

Étant donné la petite taille du labyrinthe, nous avons opté pour un Dijkstra dont les distances entre sommets des graphes seraient tous égaux à un.

### 3.1.2 Conception préliminaire

#### 1. TAD

**TAD tDirection**    **Nom** : tDirection

**Opérations** :

- **directionOuest** : tDirection → tDirection
- **directionEst** : tDirection → tDirection

**TAD Robot**

**Nom** : Robot

**Utilise** :

- tDirection
- NaturelNonNul

**Opérations** :

- **setDirection** :  $tDirection \times Robot \rightarrow Robot$
- **getDirection** :  $Robot \rightarrow tDirection$
- **getCaseRobot** :  $Robot \rightarrow NaturelNonNul$
- **setCaseRobot** :  $Robot \times NaturelNonNul \rightarrow Robot$

### **TAD Chemin**

**Nom** : Chemin

**Utilise** :

- $NaturelNonNul$

**Opérations** :

- **obtenirChemin** :  $Chemin \rightarrow \text{tableau}[1.. NB\_POINTS\_MAX]$  de  $NaturelNonNul$
- **obtenirNbPoints** :  $Chemin \rightarrow NaturelNonNul$
- **fixerNbPoints** :  $Chemin \times NaturelNonNul \rightarrow Chemin$

### **TAD File**

**Nom** : File

**Utilise** :

- $EtatDuDijkstra$
- $Naturel$

**Opérations** :

- **initialisation\_file** :  $\rightarrow File$
- **estVideFile** :  $File \rightarrow Naturel$
- **estPleineFile** :  $File \rightarrow Naturel$
- **enfiler** :  $File \times EtatDuDijkstra \rightarrow File$
- **defiler** :  $File \rightarrow File$
- **liberer\_file** :  $File \rightarrow$

**Préconditions** :  $\text{enfiler}(f, pDij) : \text{non estPleineFile}(f)$

**Préconditions** :  $\text{defiler}(f) : \text{non estVideFile}(f)$

## **2. Signatures des fonctions**

Dans cette sous partie, nous établirons les signatures de toutes les fonctions et procédures utilisées dans la partie algorithme du projet. Toutefois, pour la partie conception détaillée, nous ne présenterons que les algorithmes non triviaux et essentiels à notre résolution du problème principal : Trouver le chemin le plus court.

### **Fonctions et procédures du TAD Robot**

- Fonction  $\text{directionOuest}(\text{direction} : tDirection) : tDirection$
- Fonction  $\text{directionEst}(\text{direction} : tDirection) : tDirection$
- Fonction  $\text{getDirection}(r : Robot) : tDirection$
- Procédure  $\text{setDirection}(E/S r : Robot, E \text{ direction} : tDirection)$
- Fonction  $\text{getCaseRobot}(r : Robot) : NaturelNonNul$
- Fonction  $\text{setCaseRobot}(E/S r : Robot, E \text{ nouvelleCase} : NaturelNonNul)$

### **Fonctions et procédures du TAD Chemin**

- Fonction  $\text{obtenirNbPoints}(c : Chemin) : naturel$
- Procédure  $\text{obtenirChemin}(E c : Chemin, S \text{ tabPoints} : \text{tableau}[1..NB\_POINTS\_MAX] \text{ de } NaturelNonNul)$
- Procédure  $\text{fixerNbPoints}(E/S c : Chemin, E \text{ nombre\_points} : NaturelNonNul)$

### **Fonctions et procédures du TAD File**

- Fonction  $\text{initialisation\_file}() : File$
- Fonction  $\text{estVideFile}(f : File) : naturel$
- Fonction  $\text{estPleineFile}(f : File) : naturel$



- Procedure enfile(E/S f: File, E etat: EtatDuDijkstra)
- Procedure defiler(E/S f: File)
- Procedure liberer\_file(E f: File)

**Fonctions et procédures du graphe et de Dijkstra**

- Procedure recuperationInfoFichier(E fileNameVille: ChaineDeCaractères, S l, nb\_liaisons, nbCasesObligatoires, caseInitR0bot: NaturelNonNul, tabCasesObligatoires: tableau[1..MAX\_CASES\_OBLIGATOIRES], tabPourGraphe[NB\_POINTS\_MAX][2], orientationInitRobotDirection): File
- Fonction creationGrapheVille(tabLiasonsVille: tableau[1..NB\_POINTS\_MAX][2] de NaturelNonNul, nbLiaisons: NaturelNonNul) : G\_Graphe
- Fonction dijkstra\_avec\_exclusions(g: G\_Graphe, depart, arrivee, nb\_a\_eviter : NaturelNonNul, sommet\_a\_eviter : tableau[1..NB\_POINTS\_MAX] de NaturelNonNul) : Chemin
- Fonction dijkstra(g: G\_Graphe, depart, arrivee: NaturelNonNul)
- Procedure permutation(E arr: tableau[3] de NaturelNonNul, S permutations[6][3])
- Procedure liberer\_file(E f: File)

### 3.1.3 Conception détaillée

#### 1. Types

**Type** tDirection = {NORD, SUD, EST, OUEST }

**Type** Ordre = {AV, TG, TD, NO }

**Type** Robot = structure **Champs** :

- **caseRobot** : Naturel Non Nul
- **direction** : tDirection

**Type** Chemin = structure **Champs** :

- **points** tableau[1..NB\_POINTS\_MAX] de NaturelNonNul
- **direction** : tDirection

**Type** EtatDuDijkstra = structure **Champs** :

- **point\_actuel** : NaturelNonNul
- **point\_precedent** : NaturelNonNul
- **chemin** : Chemin

**Type** Solution = structure **Champs** :

- **ordre** : tableau[1..3] de NaturelNonNul
- **chemin\_complet** : Chemin

**Type** file = structure **Champs** :

- **elemnts** : tableau[1..MAX\_FILE] de EtatDuDijkstra
- **debut** : NaturelNonNul
- **fin** : NaturelNonNul

#### 3.1.4 Code C

### 3.2 Conclusion

#### 3.2.1 Répartition du travail

#### 3.2.2 Conclusion du projet

| <b>Tâches</b>                                           | <b>Jahnis</b> | <b>Mathieu</b> | <b>Alioune</b> | <b>Ylann</b> | <b>Ilan</b> | <b>Neil</b> |
|---------------------------------------------------------|---------------|----------------|----------------|--------------|-------------|-------------|
| Construction du robot                                   | X             | X              | X              | X            | X           | X           |
| Fonctions moteur                                        |               | X              | X              |              | X           |             |
| Fonctions capteurs                                      | X             |                |                |              |             | X           |
| Fonctions LCD                                           |               |                | X              | X            |             |             |
| Tests unitaires électronique                            | X             | X              |                |              | X           |             |
| Débogage électronique                                   |               | X              |                |              | X           |             |
| Fonctions liées à la détermination du plus court chemin | X             |                |                |              |             |             |
| Fonctions liées à la création du graphe                 |               |                |                |              |             | X           |
| Tests unitaires algorithme                              | X             |                |                |              |             | X           |
| Débogage algorithme                                     | X             |                |                |              |             | X           |
| Mise en place du makefile et bashrc                     |               |                |                | X            |             | X           |
| Rédaction du rapport                                    | X             |                |                |              |             |             |

TABLE 1 – Répartition des tâches